

UNIVERSIDAD DE LAS FUERZAS ARMADAS



“ESPE”

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN ANÁLISIS Y DISEÑO DE SOFTWARE

CARRERA DE INGENIERÍA EN SOFTWARE



NOMBRE:

- Blacio Machuca Julio César
- Sigcha Palma Kenned Jhair
- Solano Sanchez Josue Vital
- Toapanta Jama Alexander Jhosue

NRC: 30724

FECHA: 21/05/2026

DOCENTE: Ing. Jenny Ruiz Robalino

Builder:

Descripción:

Es un patrón de diseño creacional diseñado para solucionar el problema de la creación de objetos complejos, permitiendo que su proceso de construcción sea independiente de las partes que lo componen y de cómo estas se ensamblan.

Propósito e Intento Central

El objetivo principal del Builder es separar la construcción de un objeto complejo de su representación

Esto significa que un mismo proceso de construcción (el algoritmo de creación) puede dar como resultado diferentes representaciones o tipos de productos, dependiendo del "constructor" específico que se utilice.

Participantes y sus Roles

El patrón se estructura típicamente con cuatro componentes clave:

- **Builder (Constructor):** Interfaz abstracta que define las operaciones necesarias para crear las diferentes partes de un objeto producto
- **ConcreteBuilder (Constructor Concreto):** Implementa la interfaz de Builder, construye y ensambla las partes, y mantiene el rastro de la representación que está creando. Además, proporciona un método específico para recuperar el producto resultante (por ejemplo, `getVacationPlanner()` o `GetASCIIText()`)
- **Director:** Es el objeto que orquesta el proceso. Construye el objeto final utilizando la interfaz del Builder, notificando paso a paso qué partes debe añadir
- **Product (Producto):** Representa el objeto complejo en construcción. Incluye las clases que definen los componentes internos y las interfaces para ensamblarlos.

Características:

- **Construcción paso a paso:** A diferencia de otros patrones que crean productos en un solo paso, el Builder permite una construcción incremental y bajo un control más preciso del proceso.
- **Encapsulamiento de la complejidad:** Oculta al cliente los detalles internos sobre cómo se ensamblan las partes y cuál es la representación final del producto.
- **Independencia del algoritmo:** El algoritmo para crear el objeto es independiente de las partes que lo componen y de cómo estas se ensamblan.
- **Variedad de representaciones:** Permite cambiar la representación interna del producto simplemente definiendo un nuevo "ConcreteBuilder", sin alterar el algoritmo de construcción principal.
- **Control fino sobre la estructura:** Proporciona un control mucho más estricto sobre la estructura interna del producto resultante en comparación con otros patrones creacionales.

Ventajas y desventajas

Ventajas	Desventajas
Variación de la representación interna: Al utilizar una interfaz abstracta, es posible cambiar la estructura y representación interna del producto simplemente definiendo un nuevo constructor concreto, sin alterar el código del Director ni del Cliente	Mayor conocimiento del dominio por parte del cliente: En comparación con el uso de una Factory, el cliente a menudo requiere tener un mayor conocimiento de los pasos y del dominio para dirigir correctamente la creación del objeto

Aislamiento del código de construcción: El patrón mejora la modularidad al encapsular la forma en que se crea y se representa un objeto complejo. El cliente no necesita conocer las clases que definen la estructura interna del producto	Complejidad del diseño: Como ocurre con muchos patrones, su aplicación introduce clases y objetos adicionales (el Director, el Builder abstracto y múltiples Builders concretos), lo que puede aumentar la complejidad y las capas de indirección del sistema
Control más fino sobre el proceso de construcción: A diferencia de otros patrones creacionales que construyen objetos en un solo paso, el Builder construye el producto de forma incremental (paso a paso) bajo el mando del Director. Solo cuando el proceso termina, el cliente recupera el producto del constructor.	Dependencia entre productos y constructores: Aunque el algoritmo es independiente de las partes, los constructores concretos están estrechamente ligados a los tipos de productos específicos que crean
Reutilización del algoritmo de construcción: El mismo proceso de construcción (lógica en el Director) puede ser reutilizado para crear diferentes productos dependiendo del Builder concreto que se le asigne	Sobrecarga en la implementación inicial: Implementar la interfaz del Builder y sus clases derivadas puede requerir más trabajo inicial que una solución ad hoc, aunque esto suele compensarse con la flexibilidad futura
Facilidad para construir estructuras compuestas: Es un patrón ideal para crear objetos que tienen una estructura jerárquica o de árbol (como los objetos Composite).	

Modelo Builder en base al CRUD estudiante:

Diagrama de Clases

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false

package "Patrón Builder" {
    interface "IBuilderEstudiante" <<Builder>> {
        + buildId(id: int): IBuilderEstudiante
        + buildNombre(nombre: String): IBuilderEstudiante
        + buildEdad(edad: int): IBuilderEstudiante
        + getResult(): Estudiante
    }

    class "EstudianteConcreteBuilder" <<ConcreteBuilder>> {
        - estudiante: Estudiante
        + buildId(id: int): IBuilderEstudiante
        + buildNombre(nombre: String): IBuilderEstudiante
        + buildEdad(edad: int): IBuilderEstudiante
        + getResult(): Estudiante
    }

    class "Estudiante" <<Product>> {
```

```

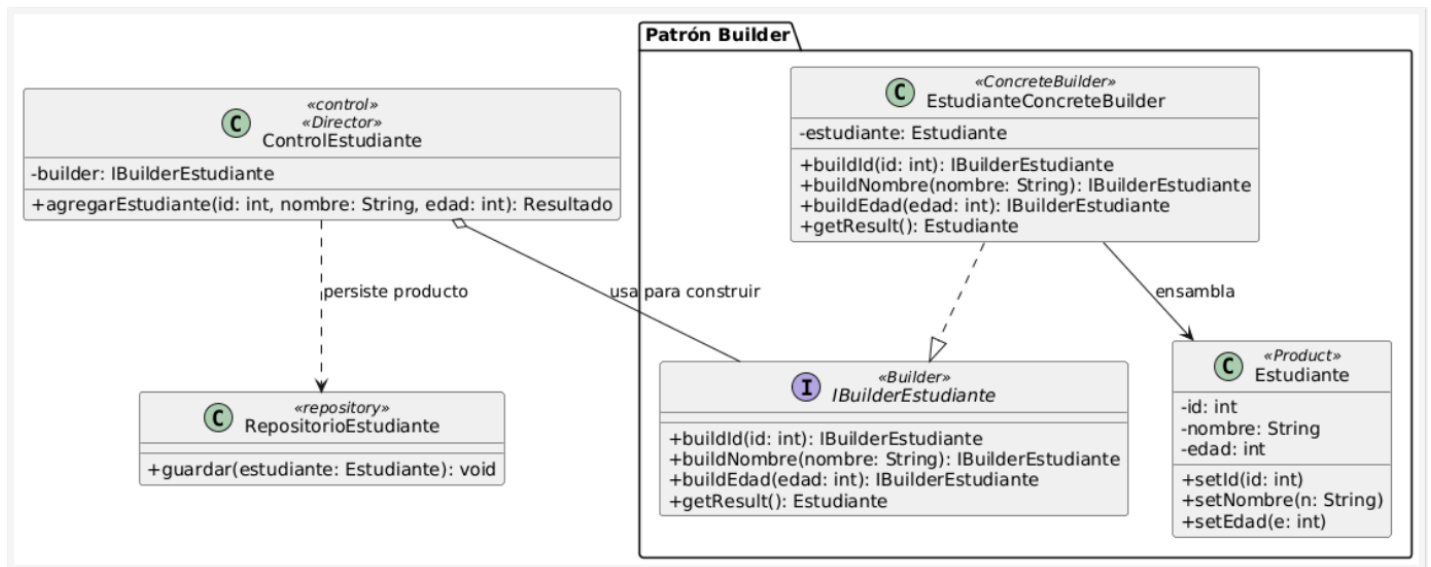
- id: int
- nombre: String
- edad: int
+ setId(id: int)
+ setNombre(n: String)
+ setEdad(e: int)
}
}

class "ControlEstudiante" <<control>> <<Director>> {
- builder: IBuilderEstudiante
+ agregarEstudiante(id: int, nombre: String, edad: int): Resultado
}

class "RepositorioEstudiante" <<repository>> {
+ guardar(estudiante: Estudiante): void
}

ControlEstudiante o-- IBuilderEstudiante : usa para construir
EstudianteConcreteBuilder ..|> IBuilderEstudiante
EstudianteConcreteBuilder --> Estudiante : ensambla
ControlEstudiante ..> RepositorioEstudiante : persiste producto
@enduml

```



Prototype:

Descripción: El patrón Prototype es un patrón de diseño creacional que especifica los tipos de objetos a crear utilizando una instancia prototípica y crea nuevos objetos copiando (o clonando) este prototipo. Es ideal utilizarlo cuando la creación de una instancia de una clase determinada es costosa o complicada. La principal ventaja de este enfoque es que el código cliente puede crear nuevas instancias sin necesidad de conocer qué clase concreta específica se está instanciando.

Características:

- **Creación por clonación:** En lugar de instanciar clases manualmente, los nuevos objetos se crean mediante la copia de una instancia existente, solicitando al prototipo que se clone a sí mismo.
- **Independencia del sistema:** Permite que un sistema sea independiente de cómo se crean, componen y representan sus productos.
- **Ocultamiento de la complejidad:** Oculta al cliente las clases de los productos concretos y las complejidades de crear nuevas instancias, lo que permite trabajar con clases específicas de la aplicación sin tener que modificarlas.
- **Reducción de subclases:** Evita tener que construir una jerarquía de clases creadoras (fábricas) que sea paralela a la jerarquía de clases de los productos.
- **Configuración y flexibilidad dinámica:** Permite agregar y eliminar productos en tiempo de ejecución de manera sencilla, además de especificar nuevos objetos variando sus valores o su estructura.
- **Eficiencia:** En ciertas circunstancias, copiar un objeto mediante este patrón puede ser mucho más eficiente y rápido que crear un objeto nuevo desde cero.

Ventajas	Desventajas
Ocultamiento de clases concretas: Oculta las clases de los productos concretos al cliente, reduciendo el número de nombres que este debe conocer y permitiéndole interactuar con clases específicas de la aplicación sin modificarlas.	Dificultad general en la implementación: El principal inconveniente es que cada subclase del prototipo debe implementar correctamente la operación de clonación (<u>Clone</u>), lo cual puede ser difícil de programar.
Adición y eliminación de productos en tiempo de ejecución: Permite incorporar una nueva clase de producto concreto en un sistema simplemente registrando una instancia prototípica en el cliente, lo que brinda una mayor flexibilidad frente a otros patrones creacionales.	Modificación de clases existentes: Integrar la operación de clonación es especialmente complicado cuando las clases a considerar ya existen previamente en el sistema.
Especificación de nuevos objetos variando valores y estructuras: Permite definir nuevos	Estructuras complejas y copias profundas: Si los objetos tienen estructuras internas

comportamientos mediante la composición de objetos (variando los valores de sus variables) o construyendo objetos a partir de subcircuitos y partes, todo esto sin necesidad de definir y programar nuevas clases desde cero.	complejas, una copia superficial (shallow copy) no es suficiente. Es necesario implementar copias profundas (deep copies) para garantizar que el clon y el objeto original sean independientes y no compartan referencias.
Reducción de subclasses: A diferencia del patrón Factory Method, que a menudo requiere una jerarquía de clases creadoras paralela a la jerarquía de productos, el uso de prototipos elimina por completo la necesidad de tener una jerarquía de clases creadoras.	Referencias circulares: La operación de clonación se vuelve particularmente engañosa y difícil cuando los objetos tienen referencias circulares en su estado interno o incluyen objetos que directamente no soportan ser copiados.

Modelo en base al CRUD estudiante:

Diagrama de Clases

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false

' Interfaz del Patrón Prototype
interface "IPrototipoEstudiante" <<interface>> {
    + clonar(): IPrototipoEstudiante
}

' Entidad que actúa como ConcretePrototype
class "Estudiante" <<entity>> {
    - id: int
    - nombre: String
    - edad: int
    + Estudiante(id: int, nombre: String, edad: int)
    + getId(): int
    + getNombre(): String
    + getEdad(): int
    + setId(id: int): void
    + setNombre(nombre: String): void
    + setEdad(edad: int): void
    + clonar(): Estudiante
}

' Clases de la arquitectura original
class "FormularioCrudEstudiante" <<boundary>> {
    - txtId: int
    - txtNombre: String
    - txtEdad: int
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + mostrarMensaje(mensaje: String)
    + mostrarTabla(estudiantes: List<Estudiante>)
```

```
}

```

```
class "ControlEstudiante" <<control>> {
- estudiantePrototipo: IPrototipoEstudiante
+ agregarEstudiante(id: int, nombre: String, edad: int): Resultado
+ actualizarEstudiante(id: int, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: int): Resultado
+ mostrarTodos(): List<Estudiante>
+ validarDatos(id: int, nombre: String, edad: int): boolean
}
```

```
class "RepositorioEstudiante" <<repository>> {
+ existeId(id: int): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: int): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: int): void
+ listarTodos(): List<Estudiante>
}
```

' Relaciones y dependencias ajustadas para evitar errores visuales
 Estudiante up.|> IPrototipoEstudiante : implementa
 ControlEstudiante o-up-> "1" IPrototipoEstudiante : clona a partir del prototipo
 ControlEstudiante ..> Estudiante : inicializa estado post-clonación
 FormularioCrudEstudiante -right-> ControlEstudiante : solicita operación
 ControlEstudiante -down-> RepositorioEstudiante : gestiona persistencia
 RepositorioEstudiante o-left- "0..*" Estudiante : almacena

@enduml

